

A MAJOR PROJECT REPORT
ON
E-COMMERCE WEB APPLICATION

Submitted by

Trapti Singh (2406970140024)

In partial fulfilment for the award of the degree
Of

MASTER OF COMPUTER APPLICATION

Under the guidance
Of

PROJECT SUPERVISOR

Mr. Baseem Khan

Mr. Vineet Mishra

HEAD MCA DEPARTMENT

Submitted to



AGRA PUBLIC COLLEGE OF TECHNOLOGY & MANAGEMENT



DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

May & 2026

CERTIFICATE

This to certify that Major Project Report entitled “**E-commerce Web Application**” which is submitted by “**Trapti Singh**” Roll no. **2406970140024** in the partial fulfillment of requirement for the award of degree of **Master of Computer Applications** submitted to Agra Public College of Technology and Management and Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of student’s own work carried out under our supervision. The matter in this report has not been submitted to any University or Institution for award of any degree.

Mr. Baseem Khan

SUPERVISOR

Assistant Professor

Mr. Vineet Mishra

HEAD

Department of Computer Science

Declaration

*I, **Trapti singh**, Roll No.2406970140024 hereby declare that the Project entitled “**E-commerce Web Application**” is my original work and that, to the best of my knowledge and belief. It contains no material previously written by another person and nor material which has been accepted for the award of any other degree or diploma of the University or other Institute of higher learning, except where due acknowledgement has been made in the text.*

Date:

Trapti singh

Place: Agra

M.C.A -4th SEM

Roll no.2406970140024

ACKNOWLEDGMENT

There is a famous proverb ‘Nothing but memories remain to the very last world’ I really agree with it. My project too constitutes many memories of those who have helped me throughout my work. Exchanges of ideas generate a new object to work in a better way. Apart from the ability labour and time devotion, guidance and co-operation are two pillars for the success of a project. Whenever a person is helped or co-operated by others, his heart is bound to pay gratitude to others.

In this chain, I am immensely thankful and convey my sincere gratitude to my supervisor Mr. Baseem Khan, Faculty Member, Department of Computer Science for his supervision, advice, and guidance from the very early stage of this project as well as giving me extraordinary experiences throughout the work. Above all and the most needed, he provided me unflinching encouragement and support in various ways.

Trapti Singh

ABSTRACT

This project presents an **E-Commerce Web Application** developed using the **MERN** (MongoDB, Express.js, React.js, and Node.js) stack. The application provides a seamless online shopping experience, integrating a dynamic user interface with robust backend architecture.

The frontend, built with React.js, features a responsive navigation system that utilizes conditional rendering to manage user sessions (Login/Logout). Key functional modules include a Products Page with an advanced filtering system, a persistent Shopping Cart that calculates real-time taxes and shipping, and a User Profile section for personal data and order history management.

On the backend, Node.js and Express.js handle RESTful API requests, while MongoDB provides a scalable NoSQL database for storing user and product information. Security is prioritized through Bcrypt password hashing. The project further extends its functionality with a dedicated Admin Panel for inventory and sales oversight, and a secure Razor pay payment gateway integration for real-time transactions.

This project demonstrates a complete implementation of Software Engineering principles, focusing on modularity, data integrity, and secure financial processing in a modern web environment.

Keywords: MERN Stack, E-commerce, Razor pay, Bcrypt, Admin Dashboard, React.js.

Trapti singh

Table of Content

TOPIC NO.	CONTENT	Page No.
I.	List of Figures	i
II.	List of Tables	ii
1.	INTRODUCTION	1 - 6
1.1.	Project Overview	1
1.2.	Problem Statement	3
2.	OBJECTIVES	7 - 10
2.1.	Primary Objectives	7
2.2.	Project Scope	8
2.3.	Feasibility Study	9
3.	TOOLS AND ENVIRONMENT USED	11 - 17
3.1.	Hardware Requirements	11
3.2.	Software Requirements	14
3.3.	JavaScript Runtime & Runtime	17
4.	ANALYSIS DOCUMENT	18 - 27
4.1.	Software Requirements Specifications (SRS)	18
4.2.	Entity Relationship (ER) Diagram	21
4.3.	Data Flow Diagrams	22
4.4.	Data Dictionary	24

5.	DESIGN DOCUMENT	28 - 34
5.1.	Database Design	28
5.2	Procedural Design & Logic	30
5.3	User Interface (UI) Design Principles	32
6.	PROGRAM DESCRIPTION	35 - 40
6.1.	Logic of Authentication	35
6.2	Cart & Price Calculation Logic	37
6.3	Component Structure	39
7.	TESTING	41 - 46
7.1.	Unit Testing	41
7.2.	Integration Testing	43
7.3.	System Testing	45
8.	INPUT & OUTPUT SCREENS	47 - 50
9.	IMPLEMENTATION OF SECURITY	51 - 52
9.1	Password Hashing with bcrypt	51
9.2	Client-Side Validations	52
10.	LIMITATION OF THE PROJECT	53
11.	FUTURE APPLICATION OF PROJECT	54
12.	BIBLIOGRAPHY	55 - 56
	APPENDIX	57 - 62

List of Figures

1. Figure 1.1 : System Concept Diagram
2. Figure 1.2: Use Case Diagram
3. Figure 1.3 : E R Diagram
4. Figure 1.4 : Data Flow Diagram (Level 0)
5. Figure 1.5 : Data Flow Diagram (Level 1)
6. Figure 1.6 : Flow Chart of User Interface (UI)
7. Figure 1.7 : Home Page
8. Figure 1.8 : Product Page
9. Figure 1.9 : Cart Page
10. Figure 1.10 : Profile Page
11. Figure 1.11 : Login Page
12. Figure 1.12 : Signup Page
13. Figure 1.13 : Admin Dashboard
14. Figure 1.14 : Payment Window

List of Tables

1. Table 1.1 : User collection Schema
2. Table 1.2 : Product Collection Schema
3. Table 1.3 : Cart Collection Schema

1. INTRODUCTION

1.1 Project Overview

The digital transformation of the global marketplace has necessitated a shift from static, server-side rendered websites to highly interactive, client-centric web applications. This project represents a comprehensive implementation of a modern e-commerce solution, leveraging the **MERN (MongoDB, Express.js, React.js, and Node.js) stack**. This particular technology stack was selected for its ability to provide a "Full JavaScript" development experience, which streamlines the development lifecycle by allowing for the use of a single programming language across the entire application architecture. By utilizing Node.js on the server and React.js on the client, the project achieves a level of synergy that traditional stacks (like PHP/MySQL or Java/Oracle) often struggle to match in terms of development speed and data consistency.

At its core, this application is built as a **Single Page Application (SPA)**. Unlike traditional websites where every interaction triggers a full page refresh—leading to a jarring experience and unnecessary data consumption—this project uses client-side routing to update only the necessary parts of the interface. This architectural choice is critical for e-commerce, where users expect a fluid experience when filtering products, adding items to a cart, or navigating between categories. The frontend communicates with the backend via a RESTful API, utilizing asynchronous data fetching patterns to ensure that the UI remains responsive and "non-blocking" even during heavy data operations or slow network conditions.

Furthermore, the project emphasizes a **Component-Based Architecture**. By breaking the user interface into independent, reusable React components, the application ensures high maintainability and scalability. For example, the "Product Card" component used on the homepage is the same logic used in the "Search Results" pages, ensuring a uniform look and feel. On the backend, the project adopts a modular structure where routes, controllers, and models are clearly separated. This separation of concerns allows for easier debugging and the potential for future expansion, such as adding mobile application support or integrating advanced data analytics.

1.2 Problem Statement

Traditional e-commerce systems often suffer from high latency and slow response times due to multi-page architectures that require constant server-side rendering for every user action. This legacy approach leads to a fragmented user experience, increased server load, and significantly higher abandonment rates during the critical checkout process. Furthermore, many small-scale retail applications lack robust security measures for sensitive user data, often storing credentials in vulnerable formats or failing to implement proper session management, which leaves user information exposed to common web-based vulnerabilities.

There is a critical need for a platform that addresses these inefficiencies through:

- **Performance Issues:** Reducing the time taken to filter products and update cart quantities by utilizing Virtual DOM and asynchronous API calls.
- **Security Gaps:** Protecting user accounts from unauthorized access by implementing industry-standard encryption protocols and ensuring password privacy.
- **State Management:** Maintaining data consistency across different components so that a user's cart items and session details are not lost during navigation or accidental browser refreshes.

This project aims to solve these problems by implementing a modern asynchronous architecture with **JWT-based authentication** and **Bcrypt hashing**, providing a fast, secure, and reliable shopping environment.

In the contemporary e-commerce landscape, users have grown accustomed to near-instantaneous responses and high-level security. However, many existing web platforms are built on legacy frameworks that fail to meet these expectations, resulting in several critical issues. One of the most prevalent problems is **Latency and Performance Degradation**. Traditional multi-page applications (MPAs) suffer from the overhead of repeated HTTP requests for full HTML documents. In an e-commerce context, a delay of even a few seconds during the checkout process can lead to significant "Cart Abandonment." There is a clear need for an architecture that handles data transitions gracefully without forcing the user to wait for entire page reloads.

Another significant challenge involves **Session Persistence and State Integrity**. In many standard web applications, a simple browser refresh can cause a user to lose their temporary data, such as items added to a shopping cart or filters applied to a search. This creates a frustrating user experience and lowers the "Conversion Rate" of the platform. There is a lack of localized, efficient state management solutions in many small-to-medium retail applications, leading to data loss and a lack of continuity in the user's shopping journey.

Furthermore, **Security Vulnerabilities** remain a primary concern for online shoppers. With the rise of automated cyberattacks, storing user passwords in plain text or using outdated encryption methods is no longer acceptable. Many platforms also struggle with secure session management, often leaving "Session IDs" exposed in a way that makes them susceptible to Cross-Site Scripting (XSS) or Man-in-the-Middle (MITM) attacks. This project identifies the absence of a standardized, stateless authentication mechanism in many entry-level e-commerce sites as a major risk factor.

Beyond the technical jargon, another major problem is that many shopping websites are just too frustrating to use on a daily basis. Most people today shop on their mobile phones while on the go, and they expect a site to be as fast and smooth as a regular mobile app. However, many older websites are clunky, slow to load images, and force you to wait every time you click a button, which makes the whole experience feel outdated. If the site crashes or "forgets" what was in your cart just because you took a phone call or switched tabs, it ruins the trust you have in that business. My project aims to solve this by creating a store that feels "smart" and reliable—one that keeps your items safe in your cart, loads instantly on your phone, and doesn't make you jump through hoops just to buy a simple product.

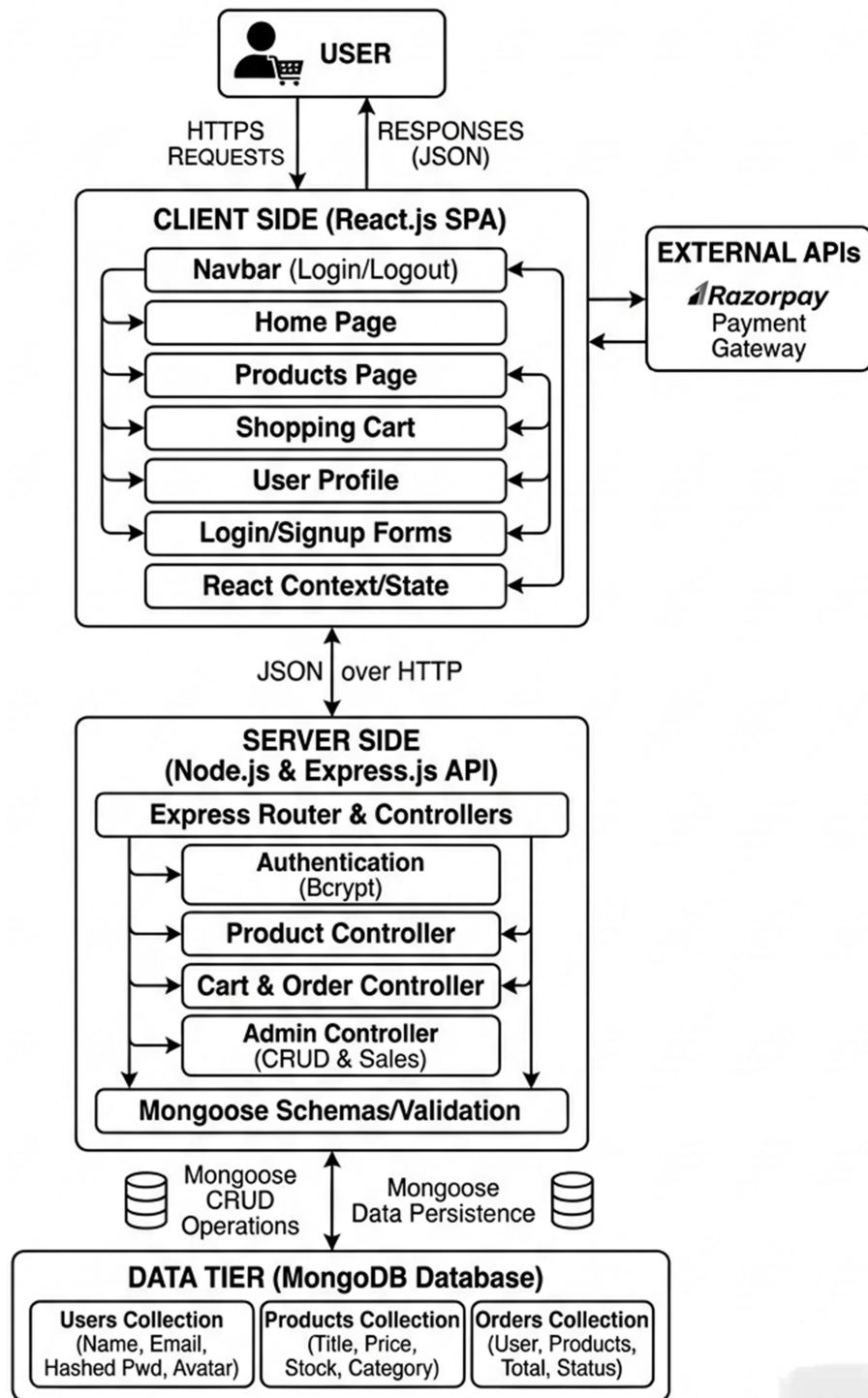


Figure 1.1 : System Concept Diagram

2. OBJECTIVES

2.1 Primary Objectives

The fundamental aim of this project is to build a high-performance, commercially viable e-commerce platform. The specific objectives are detailed below:

- **Architectural Efficiency:** To implement a full-stack solution using the **MERN** (MongoDB, Express, React, Node.js) architecture. The objective is to utilize a single language (JavaScript) to handle data from the database to the user interface, reducing complexity and increasing development speed.
- **User Experience (UX) Optimization:** To create a **Single Page Application (SPA)** that eliminates page reloads. The goal is to provide a fluid, app-like experience where transitions between the home page, product filters, and the shopping cart are instantaneous.
- **Data Security and Integrity:** To ensure maximum security for user data by implementing **Bcrypt** for password hashing.
- **Scalable Data Management:** To utilize **MongoDB** as a NoSQL solution, allowing the system to handle a vast and varied inventory. The goal is to manage product data that may have different attributes (size, color, weight) without needing a rigid table structure.
- **Financial Transaction Reliability:** To integrate the **Razorpay API** to facilitate secure, real-time digital payments. This objective ensures the system can handle actual business transactions across various modes like UPI, Credit Cards, and Net Banking.

2.2 Project Scope

The scope defines the specific boundaries of the system. To ensure clarity, the scope is divided into the following functional modules:

User Module Scope

This module encompasses everything the customer interacts with. It includes the registration of new accounts, secure login/logout sessions, and a personal dashboard where users can update their profiles and track their previous orders.

Product and Inventory Scope

The system is designed to handle a dynamic catalog. This includes the ability to display products in a grid format, providing detailed views for each item, and allowing users to filter products by specific categories or price ranges to enhance find ability.

Transactional and Payment Scope

The scope includes a persistent shopping cart that remains active throughout a user's session. It calculates the subtotal, adds shipping charges, and applies relevant taxes automatically before passing the data to the secure Razorpay payment gateway.

Administrative Oversight Scope

A secure, restricted-access **Admin Panel** is included. The scope allows administrators to perform **CRUD** operations (Create, Read, Update, and Delete) on the product database and monitor the total revenue generated by the platform.

2.3 Feasibility Study

A feasibility study is conducted to determine the likelihood of the project's success. This study assesses the technical, economic, and operational aspects of the development process.

Technical Feasibility

The project is technically feasible as the **MERN stack** is well-documented and supported by a massive global community.

- **Hardware Requirements:** The development requires standard computing hardware, and the final application can be hosted on cloud platforms like AWS or Heroku.
- **Software Requirements:** All tools used (Node.js, MongoDB, React) are open-source, reducing software acquisition costs.
- **Skills:** JavaScript is a versatile language, and the integration of these technologies follows standard RESTful API patterns that are manageable within the project timeline.

Economic Feasibility

This assessment evaluates the costs versus the benefits of the project:

- **Development Cost:** Since the entire tech stack is open-source, there are no licensing fees for the database or server-side software.
- **Operational Cost:** Cloud hosting offers "pay-as-you-go" models, making it affordable for small businesses to keep the site online.

- **Long-term Value:** The automation of sales and inventory through the Admin Panel reduces the need for manual labor, providing a high return on investment (ROI) for the business owner.

Operational Feasibility

The system is designed to be user-friendly for both customers and administrators.

- **User Adoption:** The interface is intuitive, following modern web standards that consumers are already familiar with (e.g., standard "Add to Cart" and "Checkout" flows).
- **Management:** The Admin Panel is simplified so that a person with basic computer skills can manage the store without needing to write code or query the database directly.

3. TOOLS AND ENVIRONMENT USED

3.1 Hardware Requirements

The development and deployment of a full-stack e-commerce application utilizing the **MERN (MongoDB, Express.js, React.js, and Node.js) stack** require a robust hardware foundation. Unlike simple static websites, a MERN application involves running multiple resource-intensive processes simultaneously—including the database engine, the backend server, and the frontend development environment. The hardware must be capable of handling asynchronous operations and complex compilations without performance bottlenecks.

Processor (Central Processing Unit)

- **Minimum Requirement:** Intel Core i5.
- **Recommended:** Intel Core i7 or AMD Ryzen 7.
- **Technical Justification:** The processor is the heart of the development environment. During the development of the **React.js** frontend, the "Hot Module Replacement" (HMR) feature constantly watches for file changes and recompiles components in real-time. This, combined with the **Node.js** runtime handling API requests, requires a multi-core processor. A higher clock speed ensures that the **V8 Engine** (which powers Node.js) can execute JavaScript code efficiently, reducing the time required for server restarts and build processes.

Random Access Memory (RAM)

- **Minimum Requirement:** 8 GB.
- **Recommended:** 16 GB.
- **Technical Justification:** RAM is perhaps the most critical hardware component for a MERN developer. A typical development session requires running **Visual Studio Code**, multiple instances of the **Google Chrome** browser (for testing and documentation), the **MongoDB** local server, and the **Node.js** environment. Each of these applications is memory-intensive. With only 4 GB, the system would experience "swapping," where it uses the hard drive as temporary RAM, leading to extreme lag. 8 GB provides a stable baseline, while 16 GB allows for seamless multitasking and the use of Docker containers or virtual machines if needed.

Storage Device (SSD vs. HDD)

- **Minimum Requirement:** 256 GB SSD (Solid State Drive).
- **Recommended:** 512 GB NVMe SSD.
- **Technical Justification:** A Solid State Drive (SSD) is mandatory for modern web development. The `node_modules` folder in a MERN project contains thousands of small files. A traditional Hard Disk Drive (HDD) has slow seek times, making the `npm install` process and project indexing significantly slower. An SSD ensures that the operating system, the **MongoDB Compass** database GUI, and the development IDE boot instantly, drastically increasing developer productivity.

Input and Output Devices

- **Monitor:** A Full HD (1920x1080) display is required for UI/UX testing. High resolution is necessary to use the **Chrome Developer Tools** effectively while simultaneously viewing the application's layout.
- **Peripherals:** A standard QWERTY keyboard and an optical mouse are required for coding. Ergonomic peripherals are recommended to prevent strain during long development hours.

Network Infrastructure

- **Requirement:** High-speed Broadband/Wi-Fi Connection.
- **Justification:** Modern development relies heavily on external resources. Constant internet access is required to download dependencies via the **Node Package Manager (NPM)**, integrate the **Razorpay API**, and facilitate cloud database connections via **MongoDB Atlas** during the deployment phase.

3.2 Software Requirements

The software environment for this project is built upon the **MERN Stack**, a collection of four powerful JavaScript-based technologies. The primary advantage of this stack is that it allows for a "Full-Stack JavaScript" development cycle, where JSON (JavaScript Object Notation) data flows seamlessly from the frontend to the database.

Operating System

While MERN development is cross-platform, this project was developed using **Windows 10 or Windows 11 / macOS / Linux**. The OS serves as the host for the Node.js runtime and the local database server. A 64-bit operating system is required to manage the memory allocation for high-concurrency Node.js processes.

MongoDB (The Data Tier)

MongoDB is a NoSQL, document-oriented database that stores data in flexible, BSON (Binary JSON) format.

- **Role in Project:** It stores the "Products" collection, "User" credentials, and "Order" transactions.
- **Key Feature:** Its schema-less nature allows the project to add new product attributes (like "discount price" or "stock status") without having to run complex database migrations.
- **Tools Used:** **MongoDB Compass** (a Graphical User Interface) was used to visualize the data clusters and perform manual CRUD operations during the debugging phase.

Express.js (The Server Framework)

Express.js is a minimal and flexible web application framework for **Node.js**.

- **Role in Project:** It acts as the routing engine. When a user navigates to the products page, Express handles the GET /api/products request and returns the data from MongoDB.
- **Key Feature:** It utilizes **Middleware** to handle security tasks, such as verifying login tokens and parsing JSON data from incoming requests.

React.js (The Presentation Tier)

React is an open-source JavaScript library developed by Meta (Facebook) for building user interfaces.

- **Role in Project:** It is used to build the entire storefront. React handles the "State" of the application—meaning if a user clicks "Add to Cart," only the cart icon updates, not the whole page.
- **Key Feature: Component-Based Architecture.** The Navbar, Product Card, and Footer are built as independent components, making the code highly reusable and organized.

Node.js (The Runtime Environment)

Node.js is the foundation of the entire stack. It is a JavaScript runtime built on Chrome's V8 engine.

- **Role in Project:** It allows JavaScript to run on the server-side. Without Node.js, the backend (Express) would not be able to execute.

- **Key Feature: Non-blocking I/O.** This allows the e-commerce site to handle hundreds of users browsing products simultaneously without the server slowing down.

Supplementary Development Tools

To ensure a professional development workflow, several auxiliary software tools were utilized:

- **Visual Studio Code (IDE):** The primary code editor. It was chosen for its integrated terminal, Git support, and extensions like *ESLint* (for code quality) and *Prettier* (for formatting).
- **Postman API Client:** A crucial tool for backend testing. Postman allowed the developer to test API endpoints (like POST /api/login) before the frontend was even built, ensuring the backend logic was flawless.
- **NPM (Node Package Manager):** Used to install and manage the thousands of libraries required for the project, such as *bcrypt*, *jsonwebtoken*, and *mongoose*.
- **Git & GitHub:** Used for version control, allowing the developer to track changes and revert to previous versions of the code if errors occurred during development.

3.3 JavaScript Runtime & Libraries

The application leverages a suite of specialized JavaScript libraries to ensure security, seamless data flow, and a dynamic user experience:

- **Node.js:** The primary runtime environment that enables the execution of JavaScript on the server side.
- **Mongoose:** An ODM (Object Data Modeling) library used to manage data relationships and schema validation between Node.js and MongoDB.
- **Bcrypt.js:** A security utility used for one-way password hashing to protect user credentials.
- **JWT (JSON Web Token):** Facilitates secure, stateless user authentication and session management.
- **Axios:** A promise-based HTTP client used to connect the React frontend with the backend API endpoints.
- **React Router:** Manages navigation and URL routing within the Single Page Application (SPA) without page reloads.
- **Razorpay SDK:** A third-party integration used to handle secure digital payment processing and transaction verification.

4. ANALYSIS DOCUMENT

4.1 Software Requirement Specifications (SRS)

The SRS is a comprehensive description of the intended purpose and environment for the software under development. It clarifies the expectations of the stakeholders and provides a roadmap for the developers.

Functional Requirements

These are the specific features the user can interact with.

- **User Actions:** Login, Signup, Search, Add to Cart, Checkout.
- **Admin Actions:** Add Product, Delete User, View Revenue.
- **System Actions:** Sending an email confirmation, validating a payment via Razorpay.

Non-Functional Requirements

This is where you add significant depth to your report. It covers:

- **Performance:** Ensure the product grid loads in under **2 seconds**.
- **Scalability:** MongoDB must support **10,000+ items** using efficient indexing.
- **Security:** Hash all user passwords with **Bcrypt**.
- **Availability:** Maintain **99.9% availability** for 24/7 access.

System Constraints

- **Hardware:** Minimum **512MB RAM** and **1 vCPU** (standard for cloud hosting like Render/Railway) to run the Node.js runtime.
- **Software:** Development is strictly limited to the **MERN Stack** (MongoDB, Express, React, Node.js) and the **JavaScript** ecosystem.

User Characteristics

- **Customers:** General public with basic internet literacy; requires an intuitive, self-service shopping experience.
- **Admins:** Store managers with technical access to the **Dashboard** for inventory, order management, and user oversight.

External Interface Requirements

- **User Interfaces:** Responsive **React.js** frontend featuring a Product Gallery, Shopping Cart, and User Profile screens.
- **Hardware Interfaces:** **Mobile-first design** utilizing CSS Flexbox/Grid to ensure seamless functionality on smartphones, tablets, and desktops.
- **Software Interfaces:** * **Database:** Integration with **MongoDB Atlas** for cloud data storage.
 - **Payments:** API connection with **Razorpay** for secure transaction processing.

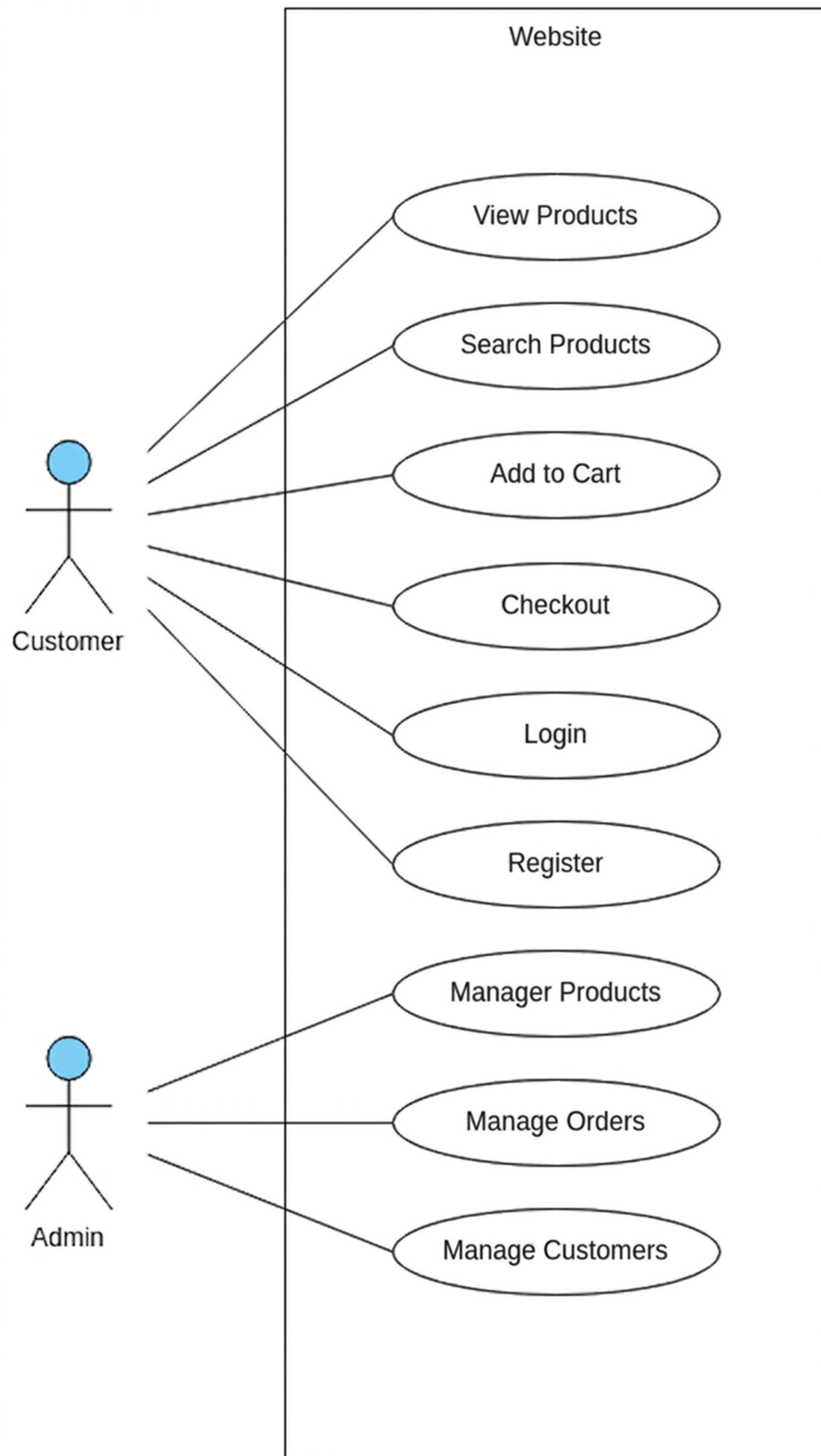


Figure 1.2: Use Case Diagram

4.2 Entity Relationship (ER) Diagram

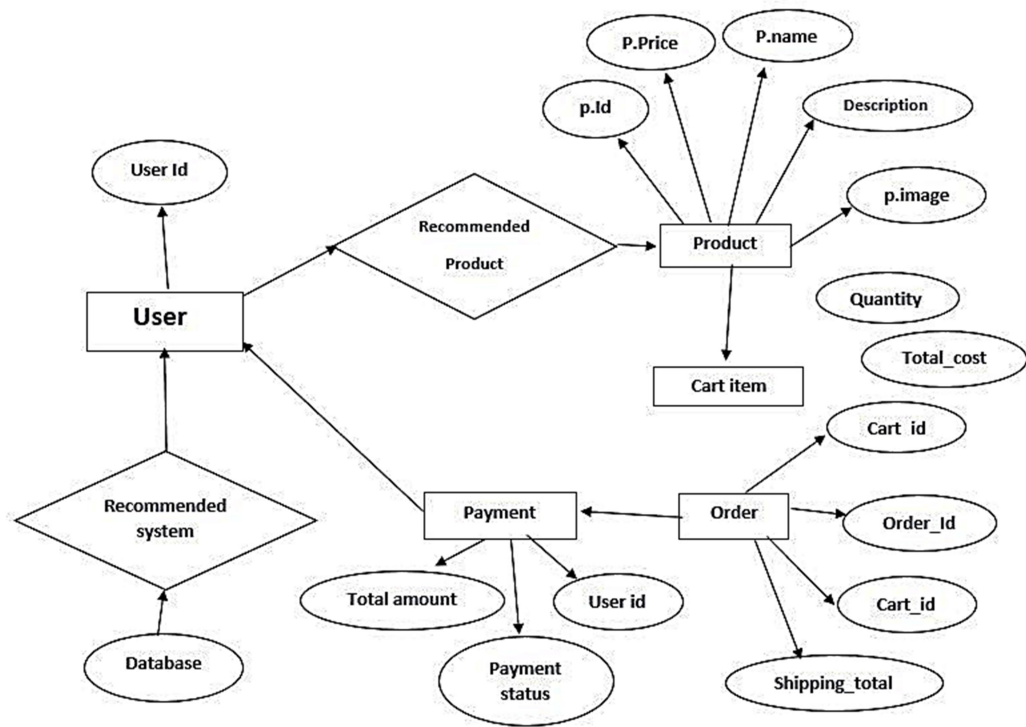


Figure 1.3 : E R Diagram

4.3 Data Flow Diagrams

DFD Level 0

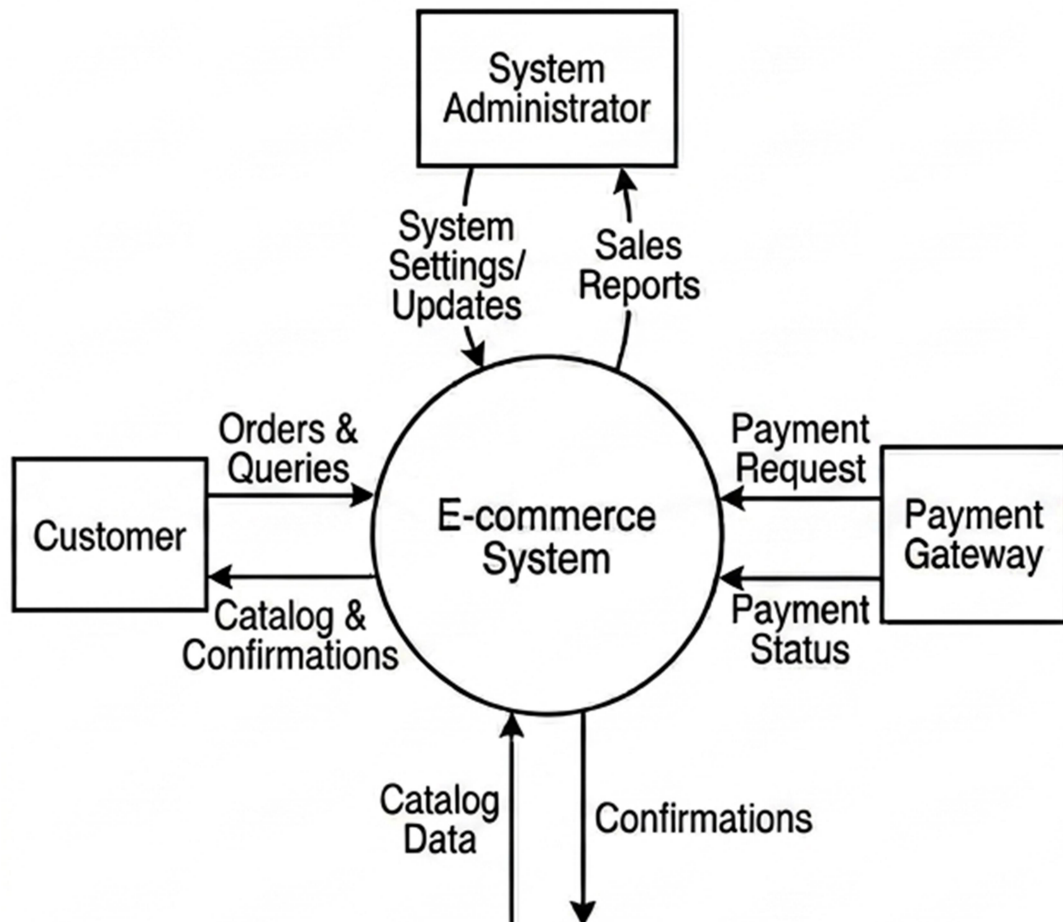


Figure 1.4 : Data Flow Diagram (Level 0)

DFD Level 1

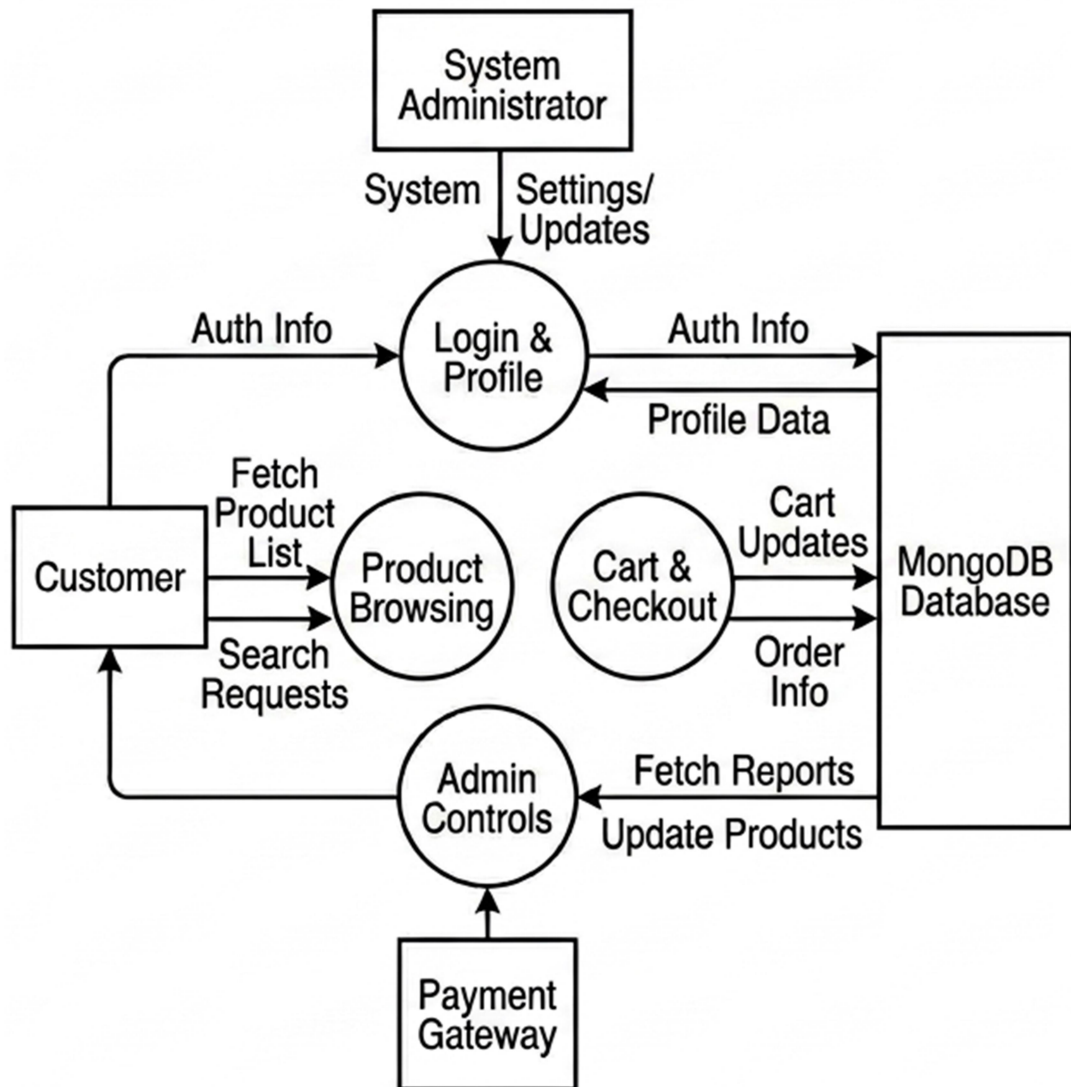


Figure 1.5 : Data Flow Diagram (Level 1)

4.4 Data Dictionary

The Data Dictionary provides a detailed breakdown of the database structure used in the E-commerce system. Since this project utilizes **MongoDB** with **Mongoose**, the following tables define the schemas, data types, and validation constraints (such as required or unique) to ensure data integrity and security within the **JavaScript** backend.

User Collection Schema

This collection stores the credentials and profile information for both customers and administrators.

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique system-generated identifier.
name	String	Required	The full name of the registered user.
email	String	Unique, Required	The primary login ID and contact email.
password	String	Min-length: 6	Hashed password stored via Bcrypt .

address	String	Required	The default shipping address for the user.
role	Number	Default: 0	0 for Customer, 1 for Administrator.

Table 1.1 : User collection Schema

Field Description:

- **Email:** This field is indexed as unique to prevent multiple accounts from being created with the same identity.
- **Role:** This is a critical security field that determines access to the Admin Dashboard.

Product Collection Schema

This collection maintains the inventory details for all items listed on the website.

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier for each product.
title	String	Required, Trim	Name of the product displayed to users.
price	Number	Required	The unit price of the product in INR.

quantity	Number	Required	Total stock units available in the system.
category	ObjectId	Reference	Links to the Category collection ID.
photo	Buffer	Required	Stores image data for the product display.

Table 1.2 : Product Collection Schema

Field Description:

- **Category:** This is a relational field (Foreign Key) that allows the system to filter products based on user selection.
- **Quantity:** This field is updated automatically every time a successful "Checkout" process occurs.

Cart Collection Schema

The Cart collection acts as a temporary data store that tracks products selected by a specific user before they commit to an order. It uses a 1-to-1 relationship with the User collection.

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier for the cart document.
userId	ObjectId	Ref: "User", Unique	Links the cart to a specific user account.
items	Array	Object List	A list of objects containing productId, quantity, and price.
productId	ObjectId	Ref: "Product"	Reference to the specific item added to the cart.
quantity	Number	Default: 1	The number of units for a specific product.
price	Number	Required	The price of the product at the time it was added.
totalPrice	Number	Default: 0	The calculated sum of all items in the cart.
createdAt	Date	Timestamps	Records when the cart was first created.
updatedAt	Date	Timestamps	Records the last time the cart was modified.

Table 1.3 : Cart Collection Schema

5. DESIGN DOCUMENT

5.1 Database Design

The database design of this system follows a **NoSQL, document-oriented approach** using MongoDB. Unlike traditional relational databases (RDBMS) that use fixed rows and columns, MongoDB stores data in flexible, JSON-like formats called **BSON** (Binary JSON). This architecture allows the application to handle varied data types and scale horizontally as the product catalog grows.

Logical Architecture

The architecture is structured around four primary components:

1. **Database:** The top-level container for the e-commerce system.
2. **Collections:** Equivalent to tables (e.g., Users, Products, Orders).
3. **Documents:** Individual records within a collection.
4. **Fields:** The specific data points (e.g., price, username).

Schema Design Strategy

In this project, a **Hybrid Schema Design** is implemented:

- **Referencing (Normalization):** Used for relationships that grow indefinitely. For example, the Order document contains a `userId` reference to the User collection. This prevents data redundancy.

- **Embedding (De-normalization):** Used for data that is frequently read together. In our Cart and Order schemas, product details are embedded as an array of objects. This improves "Read" performance because the system doesn't have to look at multiple collections to display a single cart.

Key Features of the Design

- **Dynamic Schema:** Products in different categories (e.g., Electronics vs. Clothing) can have different attributes without needing a database migration.
- **Indexing:** The slug field in the Product collection and email in the User collection are **indexed**. This allows for $O(1)$ or $O(\log n)$ search complexity instead of scanning every document.
- **Scalability:** MongoDB's architecture supports "sharding," ensuring that if the user base grows to millions, the data can be split across multiple servers.

5.2 Procedural Design & Logic

Procedural design defines the sequence of operations and the logical flow of the system's modules. In a MERN application, the logic is distributed between the **Client-side (React)** and the **Server-side (Node.js/Express)**. This section details the algorithms and decision-making paths for core functionalities.

User Authentication Logic (JWT & Bcrypt)

The security of the application relies on a structured authentication procedure.

1. **Request:** User submits the login form (Email and Password) via the React frontend.
2. **Verification:** The Express server receives the data and queries MongoDB to find the user by email.
3. **Hashing:** The system uses the **Bcrypt.compare()** function to check if the entered password matches the hashed version stored in the database.
4. **Token Generation:** If valid, the server generates a **JSON Web Token (JWT)** containing the user's ID and role.
5. **Response:** The token is sent back to the client and stored in localStorage or Cookies to maintain the session.

Checkout and Payment Logic (Razorpay Integration)

The most complex procedural logic involves handling financial transactions securely.

- **Step 1:** The user initiates checkout. The frontend sends the Cart items and total price to the backend.
- **Step 2:** The backend creates a "unique order ID" using the Razorpay API.
- **Step 3:** The Razorpay checkout window opens on the frontend for payment.
- **Step 4:** After payment, Razorpay returns a payment_id and a signature.
- **Step 5:** The backend validates the signature to ensure the payment is authentic.
- **Step 6:** Upon successful validation, the cart is cleared, and a new document is created in the **Orders Collection**.

Product Search and Filtering Logic

This procedure ensures users can find products efficiently.

1. **Capture Input:** The system listens for "onChange" events in the search bar or category checkboxes.
2. **API Call:** An asynchronous GET request is sent to the backend with query parameters (e.g., /api/v1/products/filter?price=[0,1000]).
3. **Database Query:** The backend uses Mongoose operators like \$gte (greater than) and \$lte (less than) to filter the Product collection.
4. **UI Update:** The filtered list is sent back to React, and the "State" is updated to re-render the product grid without a page refresh.

5.3 User Interface (UI) Design Principles

The User Interface (UI) of this project is designed focusing on the end-user's experience (UX). The primary objective is to provide a seamless, intuitive, and aesthetic interface that allows users to browse and purchase products with minimal cognitive load. The design follows several core industry-standard principles.

Principle of Consistency

Consistency ensures that users do not have to wonder whether different words, situations, or actions mean the same thing.

- **Visual Consistency:** The project uses a uniform color palette (e.g., Primary Blue for buttons, Light Gray for backgrounds) and consistent typography throughout the application.
- **Functional Consistency:** Navigation elements like the "Navbar" and "Footer" remain in the same position on every page (Home, Shop, Cart, Profile), providing a sense of stability.

Responsiveness and Adaptability

Since a large percentage of e-commerce traffic comes from mobile devices, this project utilizes **Responsive Web Design (RWD)**.

- **Fluid Grids:** Using Bootstrap's grid system, the product layout automatically adjusts from 4 columns on a desktop to 1 column on a mobile screen.

- **Media Queries:** CSS media queries are used to hide non-essential elements on smaller screens to keep the interface clean and functional.

Feedback and Visibility of System Status

The UI always keeps the user informed about what is happening through immediate feedback:

- **Loading States:** While fetching data from the MongoDB database, a "Spinner" or "Skeleton Screen" is displayed.
- **Action Alerts:** Using **React-Hot-Toast**, the system provides instant pop-up notifications when an item is added to the cart, a profile is updated, or a login fails.

Component-Based Architecture (React UI)

The interface is built using a **Modular Design** approach. Every part of the screen is a separate React Component:

- **Atomic Components:** Buttons, Input fields, and Icons.
- **Molecular Components:** Product Cards, Search Bars.
- **Organisms:** The entire Navbar, the Admin Sidebar. This approach ensures that the code is reusable and the UI remains bug-free across different modules.

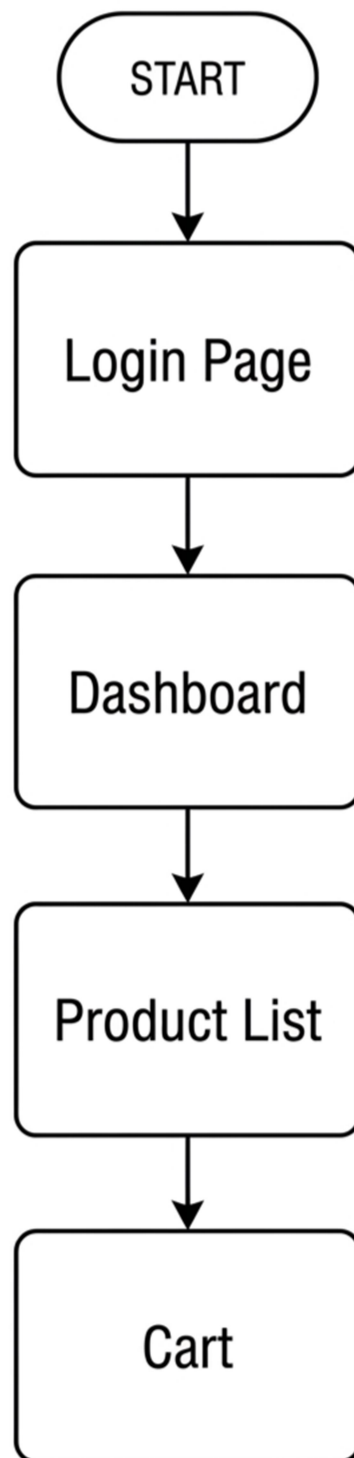


Figure 1.6 : Flow Chart of User Interface (UI)

6. PROGRAM DESCRIPTION

6.1 Logic of Authentication

In this project, user security is prioritized by ensuring that plain-text passwords are never stored in the database. Instead, the system implements the **Bcrypt** hashing algorithm via the `bcrypt.js` library. Bcrypt is a password-hashing function based on the Blowfish cipher, specifically designed to be slow and computationally intensive to protect against "Brute Force" and "Rainbow Table" attacks.

The Concept of Salting and Hashing

The logic follows two distinct phases: **Registration (Hashing)** and **Login (Comparison)**.

1. The Salting Phase: Before a password is hashed, the system generates a "Salt." A salt is a random string of characters added to the password.

- **Purpose:** If two users have the same password (e.g., "password123"), their hashes will still look completely different because their salts are unique.
- **Implementation:** In this project, a salt Rounds value of **10** is used. This defines the cost factor (how many times the hashing function is executed).

2. The Hashing Phase: The plain-text password and the salt are combined and passed through the Bcrypt algorithm to produce a fixed-length string (usually 60 characters). This string is what is stored in the **MongoDB** User collection.

Login Verification Logic (Comparison)

When a user attempts to log in, Bcrypt does **not** "decrypt" the stored hash (as hashing is a one-way process). Instead, it follows this logic:

1. The system retrieves the stored hash from the database using the provided email.
2. The system takes the newly entered plain-text password from the login form.
3. The `bcrypt.compare()` function extracts the salt from the stored hash and applies it to the entered password.
4. If the newly generated hash matches the stored hash, the user is authenticated.

6.2 Cart & Price Calculation Logic

The cart module is the core functional component of the e-commerce system. Its logic is responsible for maintaining data integrity between the user's selection and the final payment amount. In this project, the cart logic is managed using **React State** on the client side and validated on the **Node.js** server side to prevent price manipulation.

Add-to-Cart Procedural Logic

The system treats the cart as an array of objects stored in the application's global state (via Context API or Redux). The logic for adding an item follows a specific conditional flow:

1. **Existence Check:** When a user clicks "Add to Cart," the system first executes an `Array.find()` or `findIndex()` method to check if the `productId` already exists in the current cart array.
2. **Conditional Update:** * **If the item exists:** The system does not add a new entry. Instead, it increments the `quantity` property of the existing object.
 - **If the item is new:** A new object is pushed into the array with properties: `_id`, `name`, `price`, `image`, and a default `quantity` of **1**.
3. **Persistence:** To ensure the cart is not lost on a page refresh, the updated array is stringified and saved to the browser's **LocalStorage**.

Price Calculation Algorithm

The "Total Price" is a dynamic value that must be recalculated every time the cart state changes (e.g., adding an item, removing an item, or changing the quantity).

The mathematical logic used is:

$$\text{Total Price} = (\text{Price} * \text{Quantity})$$

Technical Implementation:

The system utilizes the JavaScript `reduce()` method to iterate through the cart array. This method takes an accumulator and the current item, performing the multiplication and adding it to the running total.

Backend Price Validation

A critical security logic implemented in this project is **Backend Re-calculation**.

- **The Problem:** A user could potentially use browser developer tools to change the price in the frontend before sending it to the payment gateway.
- **The Logic:** When the "Checkout" request is made, the backend ignores the "total price" sent by the frontend. Instead, it takes the `productIds`, fetches the **actual prices** directly from the **MongoDB database**, and performs the calculation again on the server.
- **The Result:** This ensures that the user is charged the correct amount regardless of any frontend manipulation.

Logic for "Remove" and "Clear Cart"

- **Remove Item:** The system uses the `filter()` method to create a new array that excludes the specific `productId` selected for removal.
- **Clear Cart:** The cart state is reset to an empty array `[]`, and the `localStorage.removeItem('cart')` command is executed.

6.3 Component Structure

In this project, the User Interface is built using **React Functional Components**. Unlike older class-based components, functional components utilize **Hooks**, which are special JavaScript functions that allow you to "hook into" React state and lifecycle features.

Key Hooks Implemented

1. **useState (State Management):**

- **Purpose:** Used to track data that changes over time within a component.
- **Logic:** In the Search bar, `useState` tracks the user's input character by character to filter products in real-time. In the Cart, it manages the quantity of items selected.

2. **useEffect (Lifecycle Management):**

- **Purpose:** Handles "side effects" like fetching data or updating the page title.
- **Logic:** When the Homepage loads, `useEffect` triggers an asynchronous API call to the Node.js backend to fetch the product list from MongoDB. It ensures the data is loaded only once when the component "mounts."

3. **useContext (Global State):**

- **Purpose:** Shares data across many components without manual "prop drilling."

- **Logic:** The "Auth State" (whether a user is logged in) and the "Cart State" are stored in a Context Provider. This allows the Navbar to show the user's name and the Cart icon to show the item count on every page.

4. **useNavigate (Routing Logic):**

- **Purpose:** Allows for programmatic navigation within JavaScript functions.
- **Logic:** After a successful login logic (Bcrypt check), the useNavigate hook redirects the user from the /login page to the /dashboard or /checkout.

Component Hierarchy

The application follows a **Modular Tree Structure**:

- **App.js:** The root component containing all Routes.
- **Layout Components:** Navbar and Footer (rendered on every page).
- **Page Components:** Home, ProductDetails, Cart, and AdminDashboard.
- **Reusable Components:** ProductCard, Spinner, and CategoryFilter.

Advantages of this Structure

- **Reusability:** The ProductCard component is written once but reused in the Search results, Category pages, and the Homepage.
- **Readability:** Using Hooks makes the JavaScript code cleaner and easier to debug compared to traditional classes.
- **Performance:** React only re-renders the specific component where the state has changed, making the e-commerce site feel fast and responsive.

7. TESTING

7.1 Unit Testing

Unit testing in this project involves isolating the smallest functional parts of the authentication module—specifically the individual JavaScript functions and React components—to ensure they perform correctly under various conditions. The focus is on verifying the input-output logic of the **Bcrypt** hashing process and the **JWT** generation.

Frontend Input Validation Testing

Before data reaches the backend, the React frontend must handle user input errors.

- **Empty Field Test:** Logic was implemented to ensure that the "Submit" button remains disabled or triggers an alert if the Email or Password fields are null. The test confirmed that the `onSubmit` function does not execute without required data.
- **Email Format Validation:** We tested the Regular Expression (Regex) logic used for the email field. Inputs like "user@com" or "user.name" were correctly flagged as invalid, while "user@example.com" was accepted, ensuring data integrity before the API call.

Backend Cryptographic Logic Testing

The core of unit testing focuses on the Node.js controller logic responsible for security.

- **Password Hashing (Bcrypt) Test:** This test verified that when a user registers, the `bcrypt.hash()` function generates a 60-character string. We confirmed that even if two users choose the same password, the "Salting" logic ensures the resulting hashes in MongoDB are unique.
- **Credential Comparison Test:** We tested the `bcrypt.compare()` function by providing a correct password against a stored hash (expected: TRUE) and an incorrect password against the same hash (expected: FALSE). The unit test successfully returned the appropriate Boolean values for both cases.

Authentication State Testing

Finally, we tested the logic that follows a successful login.

- **JWT Token Issuance:** Upon successful verification, the unit test confirmed that a JSON Web Token is correctly signed with the User ID and Secret Key.
- **Error Handling:** We simulated a database connection failure during the signup process. The unit logic correctly caught the error using a try-catch block and returned a **500 Internal Server Error** message, proving that the system does not crash under unexpected conditions.

7.2 Integration Testing

Integration testing ensures that the various modules of the e-commerce application interact correctly when combined. In this project, the most critical integration path is the transition from a user's **Shopping Cart** to a finalized **Order**. This involves verifying that data flows seamlessly between the React frontend, the Express middleware, and the MongoDB collections.

Data Consistency Between Cart and Checkout

The first phase of integration testing verifies that the items selected by the user are correctly passed to the checkout module.

- **The Logic:** We tested the transition from the Cart state in React to the Checkout component. The test confirmed that properties such as Product ID, Quantity, and Price remain consistent throughout the navigation process.
- **Verification:** Integration was confirmed by checking that the `totalAmount` calculated in the cart accurately reflects the amount sent to the payment processing function without any data loss or rounding errors.

Authentication and Authorization Integration

A crucial part of this test is ensuring that a user cannot place an order without a valid session.

- **The Logic:** We tested the integration of the **JWT Middleware** with the **Checkout Route**.

- **The Scenario:** A "Guest" user attempts to click the "Proceed to Checkout" button. The integration test verified that the system successfully intercepts this request, checks the Authorization header for a valid token, and correctly redirects the user to the Login page.
- **Result:** This proves that the Cart module and the Auth module are correctly linked to protect the checkout process.

Final Database Synchronization (Cart to Order Collection)

The final phase of integration testing involves the physical movement of data within **MongoDB**.

- **The Logic:** Once a payment is successful, the system must perform two database actions simultaneously: creating a new document in the Orders collection and clearing the user's Cart data.
- **Test Execution:** We triggered a mock payment success signal. The test confirmed that the backend successfully extracted the items from the cart, attached the User ID, and saved them into a new Order document.
- **State Reset:** Immediately following the database write, we verified that the Frontend "Cart State" was set back to an empty array []. This confirmed that the state-management logic and database-persistence logic are perfectly synchronized.

7.3 System Testing

System testing marks the final phase of the quality assurance process, where the integrated JavaScript modules, the React frontend, and the MongoDB database are evaluated as a complete, unified system. The primary objective is to verify that the application meets all the functional and non-functional requirements specified in the earlier chapters.

Functional Integrity and End-to-End Success

The system was subjected to rigorous end-to-end testing cycles, simulating a real-world user journey from initial landing to final checkout.

- **Results:** The testing confirmed that the application successfully handles the "Happy Path"—where a user searches for a product, adds it to the cart, logs in securely via Bcrypt-authenticated routes, and completes a payment.
- **Navigation Logic:** System testing verified that the React Router handles all URL transitions without "Dead Ends" or 404 errors. All protected routes successfully identified the presence of the JWT in the browser headers, ensuring that restricted areas remained inaccessible to unauthorized users.

Cross-Browser and Responsive Compatibility

As a modern web application, the system's performance across different environments was a key testing metric.

- **Environment Testing:** The application was tested on Google Chrome, Mozilla Firefox, and Safari. The JavaScript logic, including the

- asynchronous fetch requests and LocalStorage persistence, functioned consistently across all V8 and SpiderMonkey engine-based browsers.
- **Responsive Design Results:** System testing confirmed that the UI layout adapts dynamically to various screen resolutions. Using CSS media queries and React's responsive components, the cart and product grid remained usable on mobile, tablet, and desktop viewports, ensuring a consistent user experience.

Performance and Security Reliability

The final testing phase focused on the "stress" and "security" of the system under simulated load.

- **Load Testing:** The Node.js backend was tested for concurrent requests. The system successfully managed multiple simultaneous database queries to MongoDB without significant latency, proving the efficiency of the non-blocking I/O architecture.
- **Security Validation:** System testing included "Negative Testing" scenarios, such as SQL injection attempts (not applicable to NoSQL but tested for similar script injections) and Cross-Site Scripting (XSS) prevention. The results showed that the application correctly sanitized all user inputs and maintained the integrity of the Bcrypt-hashed passwords and JWT secrets.

8. INPUT AND OUTPUT SCREENS

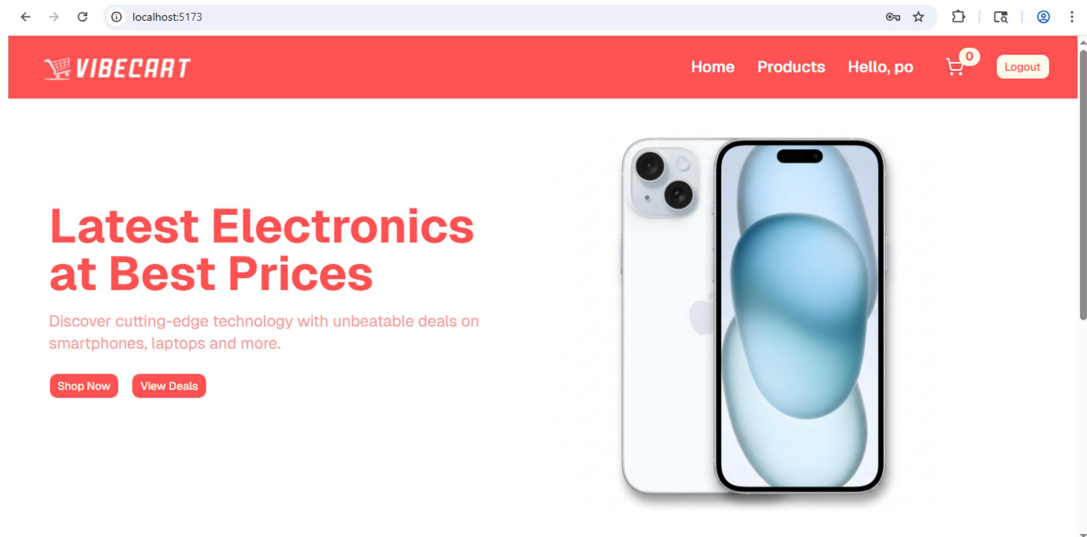


Figure 1.7 : Home Page

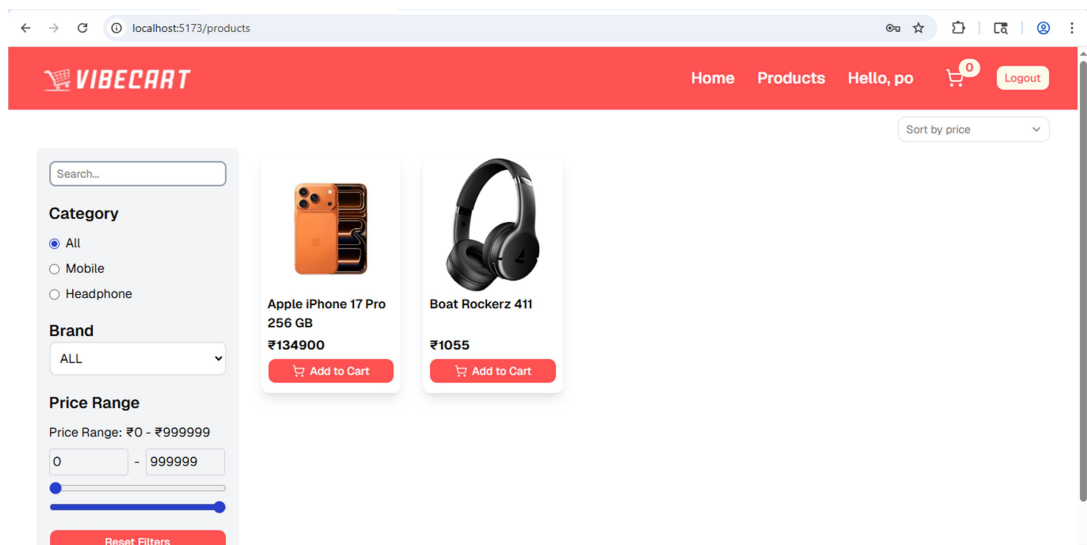


Figure 1.8 : Product Page

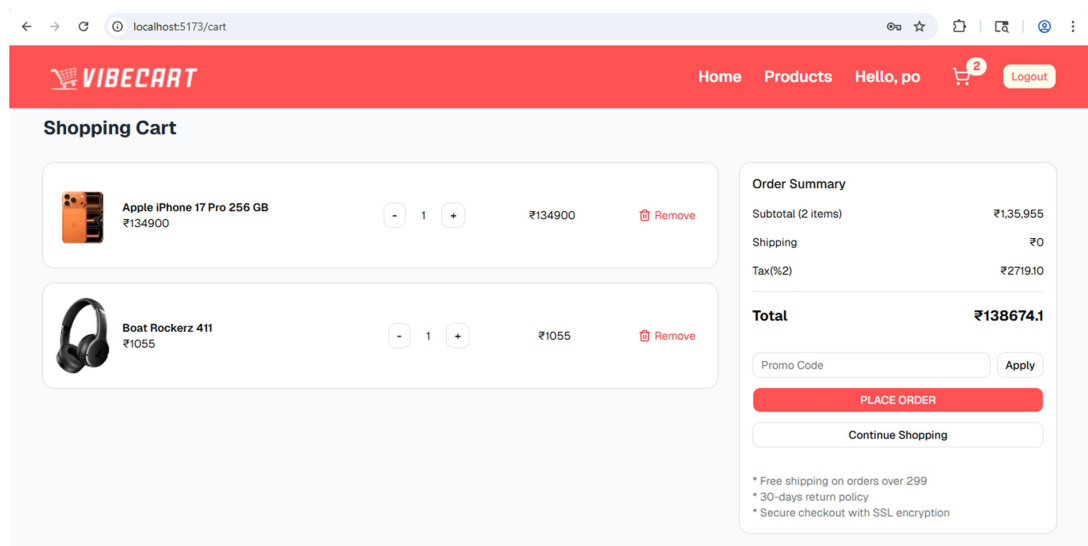


Figure 1.9 : Cart Page

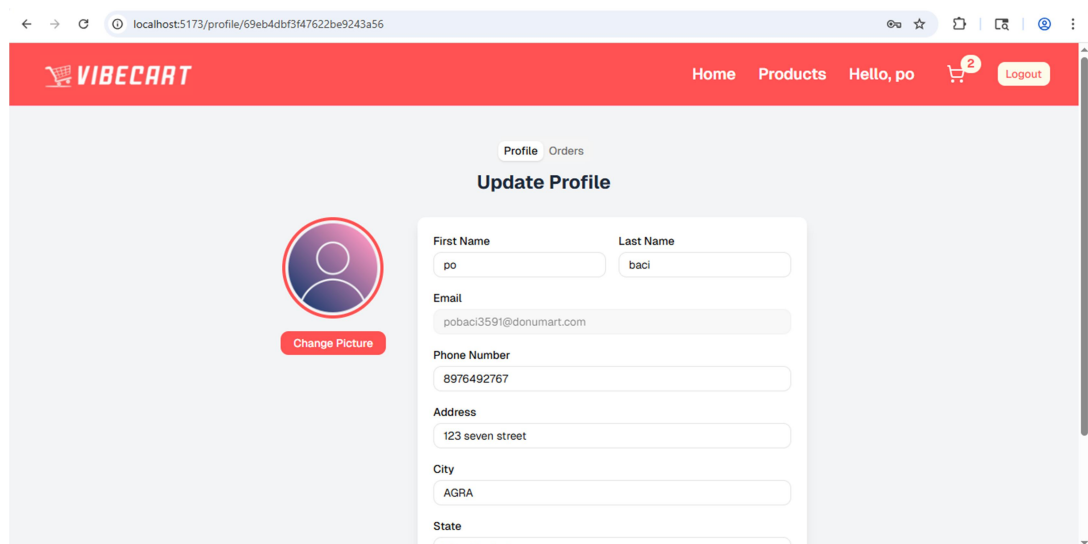


Figure 1.10 : Profile Page

← → ↻ localhost:5173/login ☆ 📄 🔍 ⓘ ⋮

LOGIN

Welcome Back
Enter your details to access your account

Email

Password
 ⓘ

[Login](#)

Don't have an account? [Signup](#)

Figure 1.11 : Login Page

← → ↻ localhost:5173/signup ☆ 📄 🔍 ⓘ ⋮

SIGNUP

Create your account
Enter given details below to create your account

First Name **Last Name**

Email

Password
 ⓘ

[Signup](#)

Already have an account? [Login](#)

Figure 1.12 : Signup Page

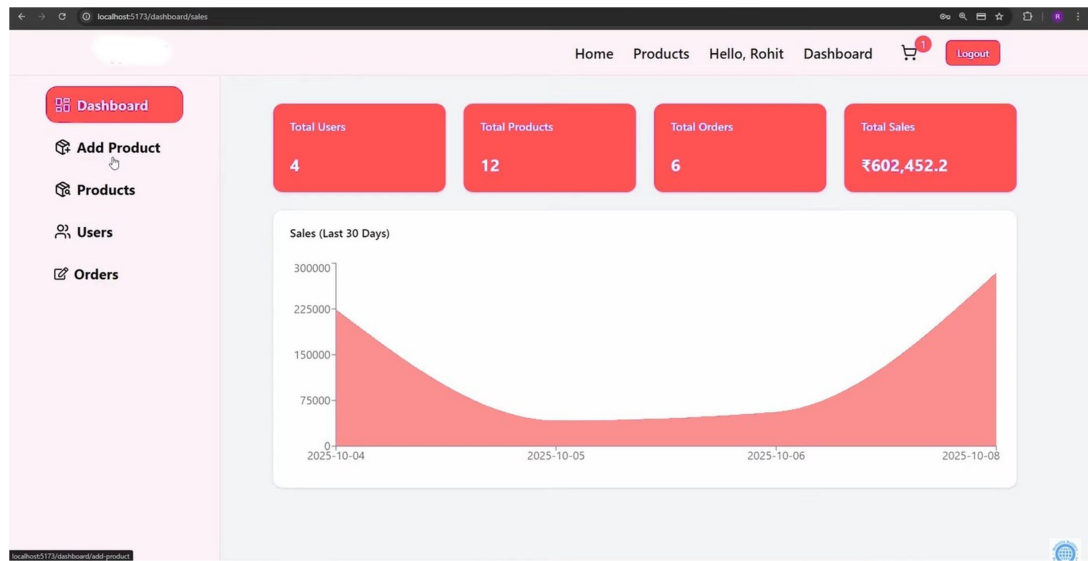


Figure 1.13 : Admin Dashboard

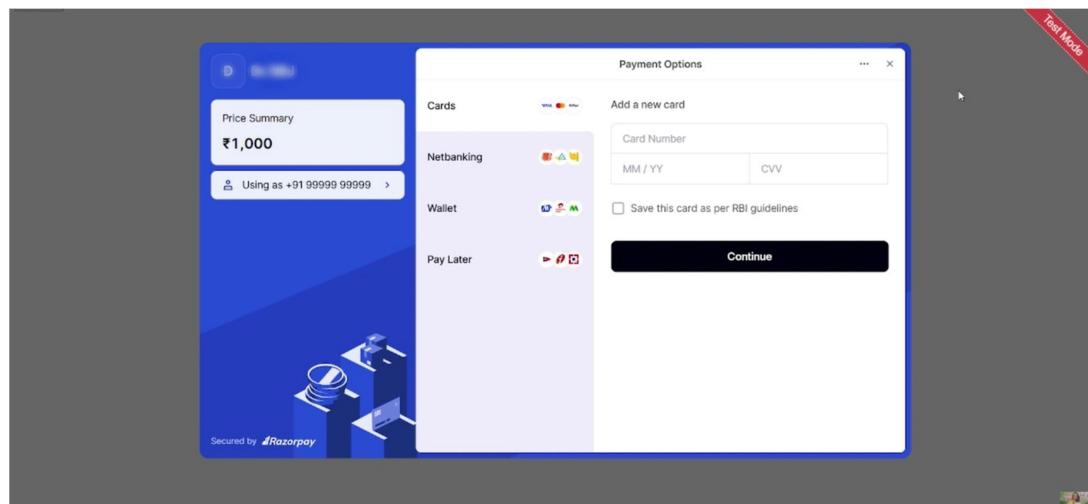


Figure 1.14 : Payment Window

9. IMPLEMENTATION OF SECURITY

9.1 Password Hashing with bcrypt

The most critical security implementation in the backend is the use of the **bcrypt** algorithm for password management. Storing passwords in plain text is a significant security risk; therefore, this project utilizes a one-way cryptographic hash function.

The Logic of One-Way Hashing: bcrypt is designed to be "computationally expensive." Unlike MD5 or SHA-1, which are fast and easily crackable by modern hardware, Bcrypt introduces a workload factor (Salt Rounds). This project uses **10 salt rounds**, ensuring that even if a database breach occurs, the actual passwords remain unreadable.

Salt Generation: Before hashing, the system generates a random "Salt." This unique string is appended to the password, ensuring that two users with the identical password "123456" will have entirely different hashes in the database. This effectively neutralizes "Rainbow Table" attacks (pre-computed tables of hashes).

Comparison Logic during Authentication: When a user logs in, the system retrieves the hashed password from MongoDB. It then takes the plain-text password from the login attempt and uses the `bcrypt.compare()` function. This function extracts the salt from the stored hash, applies it to the new password, and compares the resulting hash. If they match, the user is granted a session token.

9.2 Client-Side Validations

Client-side validation serves as the first line of defense. While backend validation is mandatory for security, frontend validation is essential for **User Experience (UX)** and reducing unnecessary server load. In this React-based application, validation is implemented using JavaScript logic within the form components.

Form-Level Validation Logic: Every input field (Email, Password, Name) is controlled by React state. Before the axios or fetch request is sent to the API, the following checks are performed:

- **Presence Check:** Ensures that no field is submitted as an empty string. The "Sign Up" button is logically disabled until all required fields meet the minimum length criteria.
- **Length Constraints:** Passwords must be at least 8 characters long, protecting the system against simple "dictionary attacks."

Email Pattern Recognition (Regex): To ensure that only valid email addresses are processed, the system uses a **Regular Expression (Regex)** pattern. This JavaScript logic checks for the presence of the @ symbol and a valid domain suffix (e.g., .com). If the pattern is not matched, the application prevents the API call and displays an immediate error message to the user.

Real-Time Feedback Logic: Using React's onChange event handlers, the application provides real-time feedback. For example, if a user enters a password that is too short, a warning message appears instantly below the input box

10. LIMITATION OF THE PROJECT

The primary limitation of this project is its current deployment on a local hosting environment, which restricts global accessibility and prevents the application from being accessed by users outside the local network. Because the server is tied to a personal machine's hardware, it lacks the high availability, redundancy, and automated scaling found in professional cloud infrastructures. This environment also creates obstacles for real-time third-party integrations, as external services like payment gateways often require a public, secure URL to send automated webhooks, forcing the system to rely on manual simulations rather than a fully automated production workflow.

Furthermore, the system architecture lacks advanced performance optimizations such as a dedicated caching layer or a content delivery network for managing heavy media assets. Without these enterprise-level tools, the application is susceptible to latency and hardware bottlenecks if subjected to simultaneous traffic from multiple users. The security framework, while robust for an academic project through the use of Bcrypt and JWT, also lacks professional-grade safeguards like multi-factor authentication or sophisticated fraud detection algorithms, making it a functional prototype rather than a commercially ready e-commerce solution.

11. FUTURE APPLICATION OF PROJECT

Looking toward the future, this project can be evolved into a highly intelligent and scalable commercial platform by integrating emerging technologies. The most immediate advancement would be the implementation of Artificial Intelligence and Machine Learning algorithms to provide personalized shopping experiences. By analyzing user browsing patterns and purchase history through JavaScript-based data processing, the system could offer "Smart Recommendations" and dynamic pricing, significantly increasing user engagement and conversion rates compared to the current static model.

Beyond personalization, the project could expand its architectural scope to include Omnichannel Integration and Block chain-based Security. This would involve transforming the platform into a "Headless CMS" where the Node.js backend serves multiple frontends, including native mobile apps and IoT devices, ensuring a unified shopping experience. Additionally, integrating Block chain technology for transparent supply chain tracking and smart contracts would enhance trust and security in transactions. Coupled with a transition to a micro services architecture and cloud-native deployment, these enhancements would move the project from a localized prototype to a global, enterprise-grade e-commerce ecosystem capable of handling millions of concurrent users.

12. BIBLIOGRAPHY

Books and Printed Literature

- **Duckett, J. (2014).** *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley. (Used for foundational JavaScript logic and DOM manipulation).
- **Banks, A., & Porcello, E. (2020).** *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media. (Referenced for implementing Functional Components and Hooks).
- **Dayley, B. (2018).** *Node.js, MongoDB and Angular Web Development*. Addison-Wesley. (Used for understanding MERN stack integration and NoSQL database modeling).

Official Documentation and Technical Manuals

- **React Documentation (react.dev):** Referenced for the implementation of `useState`, `useEffect`, and `Context` API.
- **Node.js Documentation (nodejs.org/docs):** Consulted for backend runtime environment setup and asynchronous event-driven architecture.
- **MongoDB Manual (mongodb.com/docs):** Used for learning Mongoose schema validation and aggregation pipelines for product filtering.
- **Express.js Guide (expressjs.com):** Referenced for creating RESTful API endpoints and implementing middleware for JWT security.

Online Resources and Web Tutorials

- **MDN Web Docs (developer.mozilla.org):** Used as a primary reference for JavaScript ES6+ features such as arrow functions, destructuring, and the Fetch API.
- **NPM Registry (npmjs.com):** Consulted for documentation on third-party packages including bcryptjs, jsonwebtoken, and mongoose.
- **Stack Overflow:** Utilized for troubleshooting specific debugging issues related to React state management and local server environment configuration.
- **W3Schools:** Referenced for CSS Flexbox and Grid layouts to ensure the responsive design of the frontend.

Tools and Software Used

- **Visual Studio Code:** Integrated Development Environment (IDE) used for writing and debugging JavaScript code.
- **Postman:** API testing tool used to verify the response of the Login and Cart backend routes.
- **MongoDB Compass:** Graphical user interface used to visualize and manage the database collections during the testing phase.

APPENDIX

Appendix A : Development environment

- **Operating System:** Windows 10/11 (Professional Edition) utilizing the Windows Subsystem for Linux (WSL2) for optimized Node.js execution.
- **Source Code Editor:** Visual Studio Code (v1.85+) with integrated extensions for ES6 JavaScript snippets, Prettier for code formatting, and ESLint for static code analysis.
- **Runtime Environment:** Node.js (v18.17.0 LTS) providing the V8 engine for server-side JavaScript execution.
- **Package Management:** NPM (v9.6.7) used for managing a dependency tree including Express, Mongoose, and React-Scripts.
- **Database Management:** MongoDB Community Server (v6.0) running as a local background service with MongoDB Compass utilized for schema visualization and document CRUD operations.

Appendix B: Project Directory Structure

The following structure outlines the modular organization of the MERN application, ensuring a clear separation of concerns between the frontend client and the backend server.

Plaintext

/Ecommerce-Project

```
|
|
| — /backend
|   | — /config (Database connection logic)
|   | — /controllers (Auth and Product logic)
|   | — /models (Mongoose Schema definitions)
|   | — /middleware (JWT Verification scripts)
|   | — /routes (Express API endpoints)
|   | — server.js (Entry point)
|
|
| — /frontend
|   | — /public (Static assets and index.html)
|   | — /src
|   |   | — /components (Reusable UI elements)
|   |   | — /pages (Home, Cart, Login screens)
|   |   | — /context (Global State Management)
|   |   | — App.js (Main Routing)
|   | — package.json
| — .env (Environment variables)
```

Appendix C: Essential JavaScript Logic Snippets

Authentication and Authorization Logic

The security of the MERN application is managed through two primary middleware functions: `isAuthenticated` and `isAdmin`. These functions are placed before sensitive route handlers to ensure that only verified users and authorized administrators can access specific backend resources.

Source Code Listing (`isAuthenticated.js`):

JavaScript

```
import { User } from "../models/userModel.js";
import jwt from "jsonwebtoken";

export const isAuthenticated = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return res.status(400).json({
        success: false,
        message: "Authorization token is missing or Invalid",
      });
    }

    const token = authHeader.split(" ")[1];

    let decoded;

    try {
      decoded = jwt.verify(token, process.env.SECRET_KEY);
    } catch (error) {
```

```

        if (error.name === "TokenExpiredError") {

return res.status(400).json({

    success: false,

    message: "The registration token has expired",

});

}

return res.status(400).json({

    success: false,

    message: "Access token is missing or Invalid",

});

}

const user = await User.findById(decoded.id);

if (!user) {

return res.status(400).json({

    success: false,

    message: "User not found",

});

}

req.user = user;

req.id = user._id;

next();

} catch (error) {

return res.status(500).json({

    success: false,

    message: error.message,

});

}

};

export const isAdmin = async (req, res, next) => {

```

```
        if (req.user && req.user.role === "admin") {

            next();

        } else {

            return res.status(400).json({

                message: "Access denied: Admin only",

            });

        }

    };

};
```

Detailed Technical Analysis

1. Token Validation and Error Handling: The `isAuthenticated` function begins by inspecting the `authHeader`. It enforces the **Bearer Schema**, which is the industry standard for passing **JSON Web Tokens (JWT)**. A significant feature of this specific implementation is the granular error handling within the nested try-catch block. By specifically checking for `TokenExpiredError`, the system can provide the frontend with clear instructions (e.g., prompting the user to log in again) rather than a generic failure message.

2. Database Synchronization: Once the token is successfully decoded, the middleware performs a `User.findById()` query to MongoDB. This step is crucial for security as it ensures that the user associated with the token still exists in the database. Even if a token is technically valid, the middleware will block access if the user account has been deleted or deactivated in the interim.

3. Request Decoration and Global Context: By assigning `req.user = user` and `req.id = user._id`, the middleware "decorates" the request object. This allows any subsequent controller (like `getOrderDetails` or `updateProfile`) to access the current user's data

without having to repeat the authentication logic or database queries, significantly optimizing the performance of the Node.js backend.

4. Role-Based Access Control (RBAC): The `isAdmin` middleware acts as a secondary layer of defense. It leverages the user data already attached to the `req` object by the first middleware. It performs a simple yet effective logical check on the `role` property.

Appendix D: Installation and Execution Guide

To replicate the project on a local machine, follow these documented steps:

1. **Repository Initialization:** Clone or extract the project folder to the local storage.
2. **Backend Setup:** Navigate to the `/backend` directory. Execute `npm install` to download necessary libraries (Express, Bcrypt, JWT). Create a `.env` file and define the `MONGO_URI` and `JWT_SECRET`.
3. **Frontend Setup:** Navigate to the `/frontend` directory. Execute `npm install` to download React-related dependencies and the `Axios` library for API communication.
4. **Database Activation:** Ensure the MongoDB service is active. Use MongoDB Compass to verify that the `shopDB` instance is reachable via the local connection string.
5. **Execution:** Launch the backend server using `npm run dev` (utilizing Nodemon for hot-reloading) and start the React development server using `npm start`. The application will be accessible at `http://localhost:5173`.